

Tuần 00: Nhập Môn Kiến Trúc Hướng Dịch Vụ (SOA)

Tổng Quan Môn Học, Khái Niệm Nền Tảng & Resource Naming

Giảng viên / Nhóm báo cáo: Bộ môn Công nghệ Phần mềm

Khoa Công nghệ Thông tin - Trường ĐH Công nghệ (VNU-UET)

Mục Tiêu & Kế Hoạch Bài Học (Agenda)

1. **Bản chất Kiến trúc SOA:** Tại sao monolithic chuyển dịch sang hướng dịch vụ?
2. **Nguyên lý cốt lõi:** Khớp nối lỏng (Loose Coupling) và Hợp đồng dịch vụ (Service Contract).
3. **Tư duy API-as-a-Product:** Thấp nhu cầu API và Developer Experience (DX).
4. **Mẫu thiết kế Resource Naming (JJ Geewax):** Chuẩn hóa URI danh từ, số nhiều và phân cấp.
5. **Mã nguồn minh họa:** Dịch vụ `Course Service` chuẩn RESTful API.
6. **So sánh & Đánh giá:** RPC/Chữ ký hàm truyền thống vs. Resource-Oriented Design.
7. **Định hướng thực hành:** Bộ công cụ học tập (OpenAPI, Postman, Newman, Express).

1. Khái Niệm: Kiến Trúc Hướng Dịch Vụ (SOA)

- **Định nghĩa (SOA - Service-Oriented Architecture):**

*Phong cách kiến trúc phần mềm trong đó các chức năng ứng dụng được chia thành các đơn vị dịch vụ độc lập (**Services**), cung cấp năng lực cho nhau thông qua các giao thức mạng chuẩn hóa.*

- **3 vai trò trọng yếu trong tam giác SOA:**

- **Service Provider (Bên cung cấp):** Triển khai và công bố dịch vụ.
- **Service Consumer / Client (Bên tiêu thụ):** Tìm kiếm và kích hoạt dịch vụ.
- **Service Registry / Contract (Hợp đồng):** Cầu nối xác lập luật chơi chung.

1. Khái Niệm: Khớp Nối Lỏng & Hợp Đồng Dịch Vụ

Nguyên lý vàng: Loose Coupling (Khớp nối lỏng)

- Các thành phần phụ thuộc tối thiểu vào nhau.
- Service Provider có thể thay đổi cơ sở dữ liệu từ SQL sang NoSQL, đổi ngôn ngữ từ Java sang Node.js mà **không** làm gãy Client.

Điều kiện tiên quyết: Service Contract (Hợp đồng dịch vụ)

- Hợp đồng quy định: Định dạng trao đổi (`application/json`), các tham số đầu vào, mã trạng thái HTTP trả về (`200` , `201` , `400` , `404`).
- Trong môn học này: **OpenAPI Specification (OAS 3.0.3)** là hợp đồng chuẩn.

2. Vấn Đề: Lối Mòn Thiết Kế "Ngây Thơ" (Naive Design)

Những sai lầm thường gặp khi bắt đầu làm Web API:

- **Lẫn lộn hành động vào URI:**
 - `POST /api/getCourseById`
 - `GET /api/deleteCourse?id=123`
 - `POST /api/create_new_course_record`
- **Hậu quả hệ thống:**
 - Client không thể đoán định (lack of predictability).
 - Vi phạm tính an toàn và lũy kế của HTTP (dùng `GET` để xóa dữ liệu).
 - Tài liệu API nhanh chóng trở thành "bãi rác" không thể bảo trì tự động.

2. Vấn Đề: Khủng Hoảng Trải Nghiệm Lập Trình Viên (DX)

Theo Bruno Pedro (*Building an API Product*):

- API không chỉ là cổng truyền bit; API là một **Sản phẩm (Product)** có khách hàng mục tiêu là **Lập trình viên (Developers)**.
- **Triệu chứng của một API thất bại:**
 - Không có thông báo lỗi rõ ràng (luôn trả `200 OK` kèm `{"error": "something went wrong"}`).
 - Thời gian tạo cuộc gọi đầu tiên (TTFHW - Time To First Hello World) kéo dài hàng tuần.
 - Client phải đọc mã nguồn backend mới biết payload cần truyền những gì!

3. Giải Pháp: Mẫu Thiết Kế Resource Naming (JJ Geewax)

Quy tắc chuẩn hóa danh mục tài nguyên:

1. **Dùng Danh từ số nhiều** đại diện cho tập hợp (Collection):

- `/courses` , `/students` , `/registrations`

2. **Định danh cụ thể** bằng ID trực thuộc danh từ số nhiều:

- `/courses/{courseId}` (Ví dụ: `/courses/soa-uet`)

3. **Ánh xạ hành vi qua HTTP Verbs chuẩn:**

- `GET /courses` → Liệt kê danh sách học phần (List).
- `POST /courses` → Đăng ký/Tạo mới học phần (Create).
- `GET /courses/{id}` → Xem chi tiết (Get).
- `DELETE /courses/{id}` → Hủy học phần (Delete).

3. Cấu Trúc URI & Payload Mẫu (Course Service)

Endpoint: **POST** `/api/v1/courses` (Tạo mới học phần)

- **HTTP Header:** `Content-Type: application/json`
- **Request Body:**

```
{  
  "code": "INT3111",  
  "title": "Service-Oriented Architecture",  
  "credits": 3,  
  "department": "Software Engineering"  
}
```

- **Response Status:** `201 Created`
- **Location Header:** `/api/v1/courses/65e5f1...`

4. Minh Họa Mã Nguồn: Express.js Controller

```
// POST /api/v1/courses – Resource Creation Pattern
app.post('/api/v1/courses', async (req, res) => {
  const { code, title, credits, department } = req.body;
  if (!code || !title || !credits) {
    return res.status(400).json({ error: 'Missing required fields' });
  }
  const course = await Course.create({ code, title, credits, department });
  return res.status(201).json(course);
});

// GET /api/v1/courses/:id – Single Resource Access
app.get('/api/v1/courses/:id', async (req, res) => {
  const course = await Course.findById(req.params.id);
  if (!course) return res.status(404).json({ error: 'Course not found' });
  return res.status(200).json(course);
});
```

5. So Sánh & Đánh Giá Đánh Đổi (Trade-offs)

Tiêu chí so sánh	Phong cách RPC Cổ Điển	Hướng Tài Nguyên (Resource-Oriented)
Mô hình URI	Hỗn hợp động từ (/doAction)	Thuần danh từ số nhiều (/resources)
Khả năng Cache	Kém (hầu hết dùng POST)	Tối ưu tuyệt đối với HTTP Caching (GET)
Tính nhất quán (DX)	Thấp, phụ thuộc cá nhân	Cao, tuân theo quy chuẩn toàn cầu
Hành vi phi thực thể	Tự nhiên (như gọi hàm)	Cần Custom Method (/resource:action)
Áp dụng tối ưu	Streaming, tính toán thuần	CRUD nghiệp vụ, Quản lý dữ liệu hệ thống

6. Góc Nhìn API-as-a-Product (Bruno Pedro)

Tháp Nhu Cầu API (API Hierarchy of Needs):

1. **Functionality**: API tạo và truy xuất được học phần.
2. **Reliability**: Phản hồi ổn định < 50ms, không crash server khi lỗi input.
3. **Usability**: Tên trường nhất quán (`camelCase`), mã lỗi có cấu trúc.
4. **Delight (DX)**: Cung cấp đầy đủ file OpenAPI (`openapi.yaml`) và Postman Collection sẵn kịch bản chạy thử (TTFHW < 2 phút).

"Đừng bắt người tiêu dùng dịch vụ phải đoán xem hệ thống của bạn hoạt động thế nào."

7. Khung Công Cụ Cho Sinh Viên

- **Thiết kế hợp đồng:** Viết bản đặc tả `openapi.yaml` theo chuẩn OpenAPI 3.0.3.
- **Phát triển dịch vụ:** Node.js + Express + Mongoose (nguyên lý KISS).
- **Kiểm thử tự động:**
 - Dùng Postman để khám phá trực quan.
 - Dùng Newman CLI chạy kiểm thử hồi quy trên terminal:

```
newman run postman_collection.json
```

- **Tài liệu hóa:** Render slide bằng Marp CLI.

Tổng Kết Tuần 00 & Thảo Luận

Ghi nhớ cốt lõi:

1. **SOA** là kiến trúc phân rã chức năng dựa trên **Khớp nối lỏng** và **Hợp đồng**.
2. **Resource Naming** biến URI thành danh mục tài nguyên có trật tự, dễ đoán.
3. Luôn coi **API là một sản phẩm** phục vụ lập trình viên.

Hỏi & Đáp (Q&A) - Chuẩn bị môi trường cho Tuần 01